```
class X
{
private:
    connection_info connection_details;
    connection_handle connection;
    std::once_flag connection_init_flag;

    void open_connection()
    {
        connection=connection_manager.open(connection_details);
    }
public:
    X(connection_info const& connection_details_):
        connection_details(connection_details_)
    {}
    void send_data(data_packet const& data)      <-1
    {
        std::call_once(connection_init_flag,&X::open_connection,this);   <-2
        connection.send_data(data);
    }
    data_packet receive_data()          <-3
    {
        std::call_once(connection_init_flag,&X::open_connection,this);   <-2
        return connection.receive_data();
    }
};
```

In that example, the initialization is done either by the first call to `send_data()` ❶ or by the first call to `receive_data()` ❸. The use of the member function `open_connection()` to initialize the data also requires that the `this` pointer be passed in. Just as for other functions in the Standard Library that accept callable objects, such as the constructor for `std::thread` and `std::bind()`, this is done by passing an additional argument to `std::call_once()` ❷.

It's worth noting that, like `std::mutex, std::once_flag` instances can't be copied or moved, so if you use them as a class member like this, you'll have to explicitly define these special member functions should you require them.

One scenario where there's a potential race condition over initialization is that of a local variable declared with `static`. The initialization of such a variable is defined to occur the first time control passes through its declaration; for multiple threads calling the function, this means there's the